

Pandas Cheatsheet — The Only Document You Need to Master Pandas for Data Jobs

💡 Every pandas operation you'll actually use in a data job — in one place. No theory, no fluff. Code you can copy, run, and adapt immediately.

1. Setup & Reading Data

🎯 The first five lines of every data analysis script.

```
import pandas as pd
import numpy as np

# Read data
df = pd.read_csv('file.csv')
df = pd.read_csv('file.csv', encoding='utf-8', parse_dates=[
    'date_col'])
df = pd.read_excel('file.xlsx', sheet_name='Sheet1')
df = pd.read_json('file.json')
df = pd.read_parquet('file.parquet') # fastest for
large files
df = pd.read_sql('SELECT * FROM table', con) # from a data
base connection
df = pd.read_csv('file.csv', nrows=1000) # read only f
irst 1000 rows
df = pd.read_csv('file.csv', usecols=['a', 'b', 'c']) # read
specific columns only
df = pd.read_csv('file.csv', chunksize=10000) # read in chu
nks (large files)
```

Command	What It Tells You
<code>df.shape</code>	(rows, columns)

Command	What It Tells You
<code>df.dtypes</code>	Data type of each column
<code>df.info()</code>	Column names, non-null counts, dtypes — run first on any dataset
<code>df.describe()</code>	Stats for numeric columns (min, max, mean, std)
<code>df.head(10)</code>	First 10 rows
<code>df.tail(5)</code>	Last 5 rows
<code>df.columns.tolist()</code>	All column names as a list
<code>df.sample(5)</code>	5 random rows — good for spotting patterns

2. Selecting & Filtering

🎯 The pandas equivalents of SQL SELECT and WHERE.

```
# Select columns
df['revenue']                # single column → Series
df[['region', 'revenue', 'qty']] # multiple columns → DataFrame

# Filter rows
df[df['region'] == 'North']
df[df['revenue'] > 1000]
df[df['status'].isin(['shipped', 'delivered'])]
df[df['name'].str.contains('Alice', na=False)]

# Combine conditions – ALWAYS use & | not and or
df[(df['region'] == 'North') & (df['revenue'] > 1000)]
df[(df['status'] == 'pending') | (df['status'] == 'processing')]

# Negate
df[df['region'] != 'South']
df[~df['status'].isin(['cancelled', 'returned'])]

# loc (label) vs iloc (position)
```

```

df.loc[df['revenue'] > 1000, ['region', 'revenue']] # filter rows + select cols
df.iloc[0:10, 0:3] # first 10 rows, first 3 cols
df.loc[5] # row with label 5
df.iloc[5] # 5th row by position

# query() – readable SQL-like syntax
df.query('revenue > 1000 and region == "North"')
df.query('status in ["shipped", "delivered"]')

```

3. Cleaning Data

🎯 Where 80% of analyst time goes. Master these patterns and you'll be faster than anyone.

```

# Check missing values
df.isnull().sum() # count nulls per column
df.isnull().sum() / len(df) * 100 # null % per column

# Handle missing values
df.dropna() # drop rows with ANY null
df.dropna(subset=['email', 'revenue']) # drop only if these cols are null
df.dropna(thresh=3) # keep rows with at least 3 non-nulls
df['revenue'].fillna(0) # fill with a value
df['revenue'].fillna(df['revenue'].median()) # fill with median
df['region'].fillna(method='ffill') # forward fill

```

```

df.fillna({'revenue': 0, 'region': 'Unknown'}) # fill different cols differently

# Fix data types
df['date'] = pd.to_datetime(df['date'])
df['revenue'] = pd.to_numeric(df['revenue'], errors='coerce') # bad → NaN
df['id'] = df['id'].astype(str)
df['qty'] = df['qty'].astype(int)
df['category'] = df['category'].astype('category') # memory efficient

# Clean text
df['name'] = df['name'].str.strip() # remove whitespace
df['email'] = df['email'].str.lower() # lowercase
df['region'] = df['region'].str.title() # Title Case
df['code'] = df['code'].str.replace('-', '') # remove characters
df['col'] = df['col'].str.split(',').str[0] # extract before comma

# Rename columns
df.rename(columns={'Rev': 'revenue', 'Qty': 'quantity'}, inplace=True)
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_') # clean all at once

# Remove duplicates
df.drop_duplicates() # all columns must match
df.drop_duplicates(subset=['email']) # unique by email (keep first)
df.drop_duplicates(subset=['email'], keep='last') # keep most recent

# Drop columns
df.drop(columns=['useless_col', 'another_col'])

```

4. Transforming Data

🎯 Creating new columns, applying logic, bucketing values.

```
# Create new columns
df['revenue_per_unit'] = df['revenue'] / df['quantity']
df['is_high_value'] = df['revenue'] > 1000
# boolean column
df['full_name'] = df['first_name'] + ' ' + df['last_name']
# string concat

# CASE WHEN equivalent – np.where
df['tier'] = np.where(df['revenue'] > 10000, 'VIP', 'Regular')

# Multi-condition – np.select
conditions = [
    df['revenue'] > 10000,
    df['revenue'] > 5000,
    df['revenue'] > 1000
]
values = ['Platinum', 'Gold', 'Silver']
df['tier'] = np.select(conditions, values, default='Bronze')

# apply() – use only when no vectorised option exists
df['clean_name'] = df['name'].apply(lambda x: x.strip().title())
df['score'] = df.apply(lambda row: row['a'] + row['b'], axis=1) # row-wise

# Map values
region_map = {'N': 'North', 'S': 'South', 'E': 'East', 'W': 'West'}
df['region_full'] = df['region_code'].map(region_map)

# Bin into ranges
```

```
df['age_group'] = pd.cut(df['age'], bins=[0,18,35,60,100],
labels=['Teen','Young','Mid','Senior'])
df['revenue_quartile'] = pd.qcut(df['revenue'], q=4, labels
=['Q1','Q2','Q3','Q4'])
```

5. Aggregation & GroupBy

🎯 SQL GROUP BY in pandas. Where all summary reports live.

```
# Basic groupby
df.groupby('region')['revenue'].sum()
df.groupby('region')['revenue'].mean()
df.groupby('region')['revenue'].agg(['sum','mean','count','min','max'])

# Named aggregations – cleanest syntax
df.groupby('region').agg(
    total_revenue = ('revenue', 'sum'),
    avg_revenue   = ('revenue', 'mean'),
    order_count   = ('order_id', 'count'),
    unique_cust   = ('customer_id', 'nunique'),
    max_order     = ('revenue', 'max')
).reset_index()

# Multiple group keys
df.groupby(['region', 'product'])['revenue'].sum().reset_index()

# Sort result
df.groupby('region')['revenue'].sum().sort_values(ascending=False)

# Filter groups – HAVING equivalent
grouped = df.groupby('region')['revenue'].sum()
grouped[grouped > 50000]

# transform() – keeps original row count (like SQL PARTITION)
```

```

N BY)
df['region_total'] = df.groupby('region')['revenue'].transform('sum')
df['pct_of_region'] = df['revenue'] / df['region_total'] * 100
df['region_rank'] = df.groupby('region')['revenue'].rank(ascending=False)

# Pivot table
pd.pivot_table(df, values='revenue', index='region',
                columns='product', aggfunc='sum', fill_value=0)

```

💖 6. Date & Time Operations

🎯 The most common point of confusion for beginners. These patterns cover everything.

```

# Always convert first
df['date'] = pd.to_datetime(df['date'])

# Extract date parts
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['quarter'] = df['date'].dt.quarter
df['week'] = df['date'].dt.isocalendar().week
df['weekday'] = df['date'].dt.day_name() # 'Monday', 'Tuesday'...
df['hour'] = df['date'].dt.hour

# Period truncation – for grouping
df['month_start'] = df['date'].dt.to_period('M').dt.to_timestamp()
df['quarter_start'] = df['date'].dt.to_period('Q').dt.to_timestamp()
df['year_start'] = df['date'].dt.to_period('Y').dt.to_timestamp()

```

```

# Date filters
df[df['date'] >= '2024-01-01']
df[(df['date'] >= '2024-01-01') & (df['date'] < '2025-01-01')]
df[df['date'].dt.year == 2024]
df[df['date'].dt.month.isin([1, 2, 3])]          # Q1 only

# Date arithmetic
df['days_to_ship'] = (df['ship_date'] - df['order_date']).dt.days
df['next_renewal'] = df['start_date'] + pd.DateOffset(years=1)
df['30_days_later'] = df['date'] + pd.Timedelta(days=30)

# Last N days filter
from datetime import datetime, timedelta
cutoff = datetime.today() - timedelta(days=30)
df[df['date'] >= cutoff]

# Monthly aggregation with readable labels
monthly = df.groupby(df['date'].dt.to_period('M'))['revenue'].sum()
monthly.index = monthly.index.strftime('%b %Y') # 'Jan 2024'

```

7. Merging & Joining

 SQL JOINS in pandas. Every pattern you'll need.

```

# INNER JOIN
merged = pd.merge(orders, customers, on='customer_id', how='inner')

# LEFT JOIN
merged = pd.merge(orders, customers, on='customer_id', how='left')

```



```

# Different column names
merged = pd.merge(orders, customers,
                  left_on='cust_id', right_on='customer_id', how='left')

# Multiple keys
merged = pd.merge(df1, df2, on=['customer_id', 'product_id'], how='inner')

# Anti-join – find records in left NOT in right
merged = pd.merge(orders, customers, on='customer_id', how='left', indicator=True)
no_match = merged[merged['_merge'] == 'left_only'].drop(columns='_merge')

# Stack DataFrames vertically – UNION ALL
combined = pd.concat([df_jan, df_feb, df_mar], ignore_index=True)

# Stack horizontally – add columns side by side
combined = pd.concat([df1, df2], axis=1)

# Join on index
df1.join(df2, how='left')

```

pandas <code>how=</code>	SQL Equivalent
<code>'inner'</code>	<code>INNER JOIN</code>
<code>'left'</code>	<code>LEFT JOIN</code>
<code>'right'</code>	<code>RIGHT JOIN</code>
<code>'outer'</code>	<code>FULL OUTER JOIN</code>
<code>pd.concat</code>	<code>UNION ALL</code>

8. Sorting, Ranking & Deduplication

 Ordering, Top-N, and cleaning duplicate records.

```

# Sort
df.sort_values('revenue', ascending=False)
df.sort_values(['region', 'revenue'], ascending=[True, False])

# Top N overall
df.nlargest(10, 'revenue')
df.nsmallest(5, 'revenue')

# Top N per group
df.groupby('region').apply(lambda x: x.nlargest(3, 'revenue')).reset_index(drop=True)

# Rank
df['rank'] = df['revenue'].rank(ascending=False, method='dense') # dense = no gaps
df['pct_rank'] = df['revenue'].rank(pct=True) # 0 to 1 percentile position

# Rank within group (SQL: ROW_NUMBER OVER PARTITION BY)
df['region_rank'] = df.groupby('region')['revenue'].rank(ascending=False, method='first')
top_per_region = df[df['region_rank'] == 1]

# Deduplicate
df.drop_duplicates() # exact row duplicates
df.drop_duplicates(subset=['email']) # unique per email, keep first
df.drop_duplicates(subset=['email'], keep='last') # keep most recent
df.duplicated().sum() # count duplicate rows
df[df.duplicated(subset=['email'], keep=False)] # show all duplicate rows

```

9. Window Functions

🎯 Running totals, rolling averages, lag — the most powerful pandas operations.

```
# Sort first — always required for window operations
df = df.sort_values('date')

# Running total
df['running_total'] = df['revenue'].cumsum()

# Running total per group
df['ytd_revenue'] = df.groupby(df['date'].dt.year)['revenue'].cumsum()

# Rolling average (3-period)
df['rolling_avg_3'] = df['revenue'].rolling(window=3).mean()

# Rolling within a group
df['rolling_rev'] = df.groupby('customer_id')['revenue'].transform(
    lambda x: x.rolling(3, min_periods=1).mean()
)

# LAG — previous row's value
df['prev_revenue'] = df['revenue'].shift(1)
df['prev_month'] = df.groupby('customer_id')['revenue'].shift(1)

# LEAD — next row's value
df['next_revenue'] = df['revenue'].shift(-1)

# MoM % change
df['mom_pct'] = df['revenue'].pct_change() * 100

# MoM % change per customer
df['mom_pct'] = df.groupby('customer_id')['revenue'].pct_ch
```

```

ange() * 100

# Expanding window – running aggregate from start to current row
df['cumulative_max'] = df['revenue'].expanding().max()

```

10. Reshaping Data

 Pivoting, unpivoting, melting — turning data between wide and long format.

```

# Wide → Long (unpivot) – melt
df_long = pd.melt(df,
    id_vars=['customer_id', 'region'],      # columns to keep
    value_vars=['jan_rev', 'feb_rev', 'mar_rev'], # columns to unpivot
    var_name='month',                      # new column name for headers
    value_name='revenue'                   # new column name for values
)

# Long → Wide (pivot)
df_wide = df.pivot(index='customer_id', columns='month', values='revenue')
df_wide = df_wide.reset_index().rename_axis(None, axis=1)

# Pivot with aggregation – handles duplicate index values
df_wide = df.pivot_table(
    index='customer_id',
    columns='month',
    values='revenue',
    aggfunc='sum',
    fill_value=0
)

```

```
# Explode – one row per list item
df['tags'] = df['tags'].str.split(',') # convert to list
df_exploded = df.explode('tags')

# Stack / Unstack
df.set_index(['region', 'product']).unstack('product') # product → columns
df.stack() # columns → rows
```

11. String Operations

🎯 Every `.str.` operation you need for text column cleaning.

```
# Basic text
df['col'].str.lower()
df['col'].str.upper()
df['col'].str.title()
df['col'].str.strip() # remove leading/trailing spaces
df['col'].str.strip('$') # remove specific character
df['col'].str.replace('old', 'new')
df['col'].str.replace(r'\s+', ' ', regex=True) # collapse multiple spaces

# Extract and split
df['col'].str.split(',') # split to list
df['col'].str.split(',', expand=True) # split to separate columns
df['col'].str.split(',').str[0] # get first element after split
df['username'] = df['email'].str.split('@').str[0]
df['domain'] = df['email'].str.split('@').str[1]
df['col'].str.extract(r'(\d+)') # extract regex pattern
```

```

df['col'].str.slice(0, 5) # first 5 characters

# Check / filter
df['col'].str.contains('keyword', na=False)
df['col'].str.startswith('ABC')
df['col'].str.endswith('.csv')
df['col'].str.len() # string length
df['col'].str.count(',')

# Useful combinations
df[df['email'].str.contains('@gmail.com', na=False)] # filter Gmail users
df['initials'] = df['name'].str[0] # first character

```

12. Exporting Results

🎯 Getting your output out — to CSV, Excel, clipboard, or multiple sheets.

```

# Export to CSV
df.to_csv('output.csv', index=False) # index=False removes row numbers
df.to_csv('output.csv', index=False, encoding='utf-8-sig')
# for Excel compatibility

# Export to Excel – single sheet
df.to_excel('output.xlsx', sheet_name='Summary', index=False)

# Export to Excel – multiple sheets (CORRECT way)
with pd.ExcelWriter('report.xlsx') as writer:
    summary.to_excel(writer, sheet_name='Summary', index=False)
    detail.to_excel(writer, sheet_name='Detail', index=False)

```

```

monthly.to_excel(writer, sheet_name='Monthly', index=False)

# Paste to clipboard – then Ctrl+V into Excel
df.to_clipboard(index=False)

# Export to Parquet – best for large files
df.to_parquet('output.parquet', index=False)

# Export to JSON
df.to_json('output.json', orient='records', indent=2)

# Export to SQL table
df.to_sql('table_name', con=engine, if_exists='replace', index=False)

```

Performance & Best Practices

🎯 The habits that separate junior pandas users from senior ones.

Habit	Why It Matters
Run <code>df.info()</code> first	Instantly reveals nulls, wrong dtypes, and row count
Use <code>pd.to_numeric(errors='coerce')</code>	Bad values become NaN instead of crashing the script
Use <code>&</code> and <code>\</code>	<code>, never</code> and <code>/</code> or <code>`</code>
Wrap each filter condition in <code>()</code>	Without parentheses, operator precedence breaks filters
Use <code>errors='coerce'</code> with type conversions	Prevents crashes on dirty data
Use <code>reset_index()</code> after groupby	Groupby returns a MultiIndex — reset makes it a clean DataFrame
Use <code>pd.ExcelWriter</code> for multi-sheet	Calling <code>to_excel()</code> twice overwrites — writer stacks them
Use <code>transform()</code> for group-level metrics on raw data	<code>agg()</code> collapses rows; <code>transform()</code> keeps every row

Habit	Why It Matters
Never use <code>apply()</code> on numeric columns	It's a Python loop — 10–100× slower than vectorised operations
Use <code>category</code> dtype for low-cardinality strings	Cuts memory by 5–10× on large string columns

♥ Quick Reference — One-Liners

Task	Code
Read CSV	<code>pd.read_csv('f.csv')</code>
Inspect	<code>df.info()</code> , <code>df.describe()</code> , <code>df.head()</code>
Filter rows	<code>df[df['col'] > value]</code>
Select columns	<code>df[['a','b','c']]</code>
Rename columns	<code>df.rename(columns={'old':'new'})</code>
Drop column	<code>df.drop(columns=['col'])</code>
Fill nulls	<code>df['col'].fillna(0)</code>
Drop nulls	<code>df.dropna(subset=['col'])</code>
Fix dates	<code>pd.to_datetime(df['col'])</code>
Fix numbers	<code>pd.to_numeric(df['col'], errors='coerce')</code>
Group & sum	<code>df.groupby('col')['val'].sum()</code>
Multi-agg	<code>df.groupby('col').agg(n=('id','count'), total=('rev','sum'))</code>
Running total	<code>df['rev'].cumsum()</code>
Lag	<code>df['rev'].shift(1)</code>
Rolling avg	<code>df['rev'].rolling(3).mean()</code>
MoM %	<code>df['rev'].pct_change() * 100</code>
Merge left	<code>pd.merge(a, b, on='id', how='left')</code>
Stack rows	<code>pd.concat([df1, df2], ignore_index=True)</code>
Sort	<code>df.sort_values('col', ascending=False)</code>
Top 10	<code>df.nlargest(10, 'col')</code>
Dedup	<code>df.drop_duplicates(subset=['email'])</code>
Rank	<code>df['col'].rank(ascending=False, method='dense')</code>
Melt (wide → long)	<code>pd.melt(df, id_vars=['id'], value_vars=['a','b'])</code>

Task	Code
Pivot	<code>df.pivot_table(index='a', columns='b', values='v', aggfunc='sum')</code>
Export CSV	<code>df.to_csv('out.csv', index=False)</code>
Export Excel	<code>df.to_excel('out.xlsx', index=False)</code>
Multi-sheet	<code>pd.ExcelWriter('out.xlsx')</code> context manager

✓ You're Ready When...

- You run `df.info()` and `df.isnull().sum()` before touching any new dataset
- You use `&` and `|` inside filters without thinking — never `and` / `or`
- You wrap filter conditions in `()` by reflex
- You know `agg()` collapses rows and `transform()` keeps them — and use each correctly
- You write `pd.to_numeric(errors='coerce')` for any numeric column from an external file
- You use `pd.ExcelWriter` for multi-sheet exports without looking it up
- You reach for `np.where` or `np.select` before ever writing `apply(lambda...)`
- You can load any messy file, clean it, aggregate it, and export it in one script
- You've automated at least one monthly report that used to take 30 minutes manually